PROJET SENTIMENT-APP

Compte rendu global

Travaux pratiques 1 a 6

REALISE EN BINOME PAR

Hafsa Boutayeb

et

Aya Soulaimane

Depot : hafsaboutayeb/sentiment-app

28 octobre 2026

1. Le besoin et son cadrage

Le projet fil rouge consiste à aider une équipe de relation client à traiter plus rapidement les avis de consommateurs. Le besoin retenu est de repérer les avis négatifs afin qu'ils puissent être examinés en priorité, sans laisser le modèle prendre seul une décision importante. Le système doit pouvoir signaler quand il hésite, et un agent reste responsable du traitement final.

Au TP1, nous avons comparé trois cas. Pour les ordonnances incomplètes, la fiche retient des règles métier : les champs réglementaires sont connus et une erreur peut avoir une conséquence grave sur la santé. Pour répondre aux questions sur un règlement, la fiche retient le RAG, car il faut rattacher chaque réponse à un article vérifiable. Pour prioriser les avis négatifs, nous avons retenu le machine learning. Les avis peuvent être étiquetés à partir des notes et traités en volume ((a) et (c)); une erreur de tri est réversible et l'agent peut corriger le verdict ((d)). La vérifiabilité ((b)) reste importante : les scores et les cas incertains doivent être visibles plutôt que présentés comme des certitudes.

Ce cadrage a limité le périmètre : construire un classifieur de texte francophone, mesurer ses erreurs, l'exposer par une API, puis fournir une interface et un paquet réutilisables. Le modèle sert à prioriser, pas à remplacer le jugement humain.

2. Architecture et gestion des fichiers

Le dépôt a évolué sans remplacer les étapes précédentes. `src/` contient les scripts d'entraînement, d'évaluation et de démonstration. `sentiment_app/` contient depuis le TP6 le paquet installable et son prédicteur. `api/` expose les routes FastAPI; `ui/` contient le client HTTP, les fonctions de traitement et la page Streamlit. `tests/` vérifie le cadrage, le modèle, l'API, l'interface, les limites d'entrée et le packaging. `docs/` conserve les fiches et les comptes rendus des séances.

Le code, les tests, les documentations, `data/jeu_or.csv` et `models/metrics.json` sont versionnés. Le fichier de métriques est petit et permet de relire la mesure associée à un entraînement. À l'inverse, `models/sentiment.joblib` et `data/demo_reviews.csv` sont des artefacts reconstructibles, ignorés par Git; le modèle peut être recréé avec le script d'entraînement. Le modèle n'est pas dans la roue non plus : son chemin se configure avec `SENTIMENT_MODELE`. Cette séparation évite de versionner un fichier binaire lourd et lie explicitement le modèle à la version de scikit-learn qui l'a produit.

3. Données, modèle et résultats mesurés

Le jeu de démonstration généré par `src/make_demo.py` contient 290 avis, dont 145 positifs et 145 négatifs. Le découpage reproductible réserve 20 % des avis au test : 232 pour l'entraînement et 58 pour l'évaluation. Le pipeline transforme les textes avec TF-IDF en unigrammes et bigrammes, puis entraîne une régression logistique. Le découpage est fait avant l'apprentissage du vocabulaire pour ne pas exposer au modèle les textes du test. La graine 42 permet de reproduire la séparation.

Jeu évalué	Avis du jeu	Avis de test	Vocabulaire	F1 macro	Exactitude	Rappel négatif
Démonstration	290	58	316 termes	0,879	0,8793	0,931
Allociné	Non renseigné dans les métriques versionnées	—	—	—	—	—

`models/metrics.json` indique que le modèle mesuré et conservé dans le dépôt est celui du jeu de démonstration. Le code sait charger Allociné, et le marche-à-suivre décrit cette expérience, mais aucun résultat Allociné propre à notre dépôt n'est enregistré dans les métriques actuelles. Nous ne recopions donc pas le score indicatif du support de TP comme s'il s'agissait d'une mesure faite par notre binôme.

La matrice de confusion du test de démonstration, avec les lignes comme sentiments réels et les colonnes comme prédictions, est :

Réel \ prédiction	Négatif	Positif
Négatif	27	2
Positif	5	24

Sur les 29 avis réellement négatifs, 27 sont retrouvés et 2 sont manqués; le rappel négatif est donc 27/29, soit 0,931. Sur les 29 positifs, 24 sont reconnus et 5 sont envoyés à tort dans la classe négative. Au total, 51 avis sur 58 sont bien classés, ce qui correspond à l'exactitude de 0,8793. Le F1 macro de 0,879 résume les deux classes sans laisser la seule exactitude masquer leurs erreurs.

4. Analyse des erreurs et abstention

Le compte rendu du TP2 relève sept erreurs sur les 58 avis de test et les associe aux formulations mitigées avec retournement. TF-IDF représente les mots et leurs associations locales, mais ne comprend pas toujours que la seconde partie d'une phrase annule la première. Par exemple, un avis qui commence par « produit conforme » puis conclut par « sauf que […] produit cassé » contient des indices des deux classes; l'ordre et la conclusion changent pourtant son sens. Les mots inventés posent une autre limite : ils ne contribuent pas au vocabulaire appris.

Le TP4 a ajouté le verdict `INDETERMINE` lorsque la confiance est sous le seuil de 0,65. L'abstention ne rend pas le classifieur plus intelligent, mais évite de forcer une réponse lorsque les probabilités sont trop proches ou que le texte est hors vocabulaire. Le compte rendu de la séance estime qu'environ 15 % des avis sont alors renvoyés à une vérification humaine; ce chiffre est une estimation notée dans le compte rendu TP4 et non une valeur réinscrite dans `metrics.json`.

Le jeu d'or `data/jeu_or.csv` complète le test aléatoire avec des exemples écrits pour couvrir les négations, l'ironie, les avis neutres et les retournements. Il conserve notamment l'avis négatif « À éviter absolument, très mauvaise qualité », déclaré comme erreur connue dans le test. Cet exemple montre qu'un mot positif isolé peut induire le modèle en erreur. En production, la priorité est donc de faire vérifier les avis à risque, d'examiner les faux négatifs et d'enrichir les données avec des formulations variées, plutôt que d'augmenter artificiellement le score sur le petit jeu de démonstration.

5. API et interface utilisateur

Le TP3 a séparé le modèle du transport : `POST /predict` reçoit un texte et renvoie une étiquette, un score et un aperçu; `/health` indique si l'API et son modèle sont disponibles. Pydantic valide les entrées, et l'API distingue un texte refusé (422), un modèle indisponible (503) et une erreur interne (500). Le modèle est chargé au démarrage, plutôt qu'à chaque requête.

Le TP5 ajoute un client HTTP et une page Streamlit pour saisir un avis, lire son verdict et sa confiance, et afficher une consigne humaine en cas d'abstention. Les fonctions de `ui/helpers.py` savent aussi traiter un tableau CSV et produire un résumé; cependant, dans l'état du dépôt décrit ici, la page `ui/app.py` n'expose que l'analyse d'un avis individuel. Nous ne présentons donc pas le téléversement CSV comme une fonction terminée de cette page.

Le TP6 transforme le projet en paquet installable versionné `0.6.0`. La commande `sentiment` fournit les fonctions `predict`, `serve`, `ui` et `--version`. L'API importe `sentiment_app.predict` directement, sans modifier `sys.path`; une couche de compatibilité préserve les anciens imports. La roue inclut le code de l'application et ses dépendances, mais pas les tests, les données ou le modèle. La CI construit la roue, l'installe dans un environnement isolé et effectue une prédiction avec un chemin de modèle externe.

6. Difficultés rencontrées et contrôles

Au début du TP1, les tests signalaient les fiches de cadrage encore vides. Après leur remplissage, les anciens runs rouges n'ont pas changé de couleur : ils restent des enregistrements historiques. Un run d'octobre a aussi échoué parce qu'à cette date l'historique ne contenait pas encore le commit du second membre. L'historique actuel contient des commits de Hafsa Boutayeb et d'Aya Soulaimane, et le test vérifie désormais les deux auteurs.

Le premier run du TP6 a révélé une fuite d'état dans le test du modèle absent : `monkeypatch.setattr` ne restaurait pas comme prévu la propriété dynamique `app.state.classifier` de Starlette. Le test modifie maintenant le dictionnaire d'état avec `monkeypatch.setitem`, ce qui restaure correctement la valeur après le test. Le run TP6 suivant et le run après renommage du job CI sont verts. L'historique GitHub conserve néanmoins les exécutions échouées #1, #2, #8 et #16; elles ne peuvent pas être transformées rétroactivement en réussites.

Au contrôle local du 28 octobre, avec le modèle généré pour les tests, `pytest -q` a donné **92 passed et 1 xfailed**. Le xfail est l'erreur du jeu d'or explicitement déclarée et attendue, et non une panne silencieuse. Ruff et mypy ont également passé leurs contrôles; la CI finale de la branche `main` est verte. Les résultats d'Actions les plus récents, plutôt que la couleur d'un ancien run, indiquent l'état courant du dépôt.

7. Ce que nous ferions autrement

Nous mesurerions Allociné et le jeu de démonstration dans des exécutions distinctes, en conservant pour chacune les métriques, la matrice de confusion et la version des dépendances. Nous compléterions ensuite la page Streamlit pour proposer réellement l'import, le résumé et le téléchargement d'un CSV, puis ajouterions des tests d'interface pour ce parcours. Pour la qualité du modèle, nous testerions une représentation tenant mieux compte du contexte et de la négation, sans retirer le seuil de confiance ni la validation humaine. Enfin, nous garderions un jeu de test représentatif des avis réellement reçus, anonymisé et surveillé dans le temps.

8. Répartition et outils

Membre	Éléments visibles dans l'historique Git
Hafsa Boutayeb	Commits de structure, modèle, API, tests, interface, packaging et comptes rendus; l'historique la crédite de la majorité des commits.
Aya Soulaimane	Commit du tableau du binôme dans le README; son compte est présent parmi les auteurs du dépôt.

L'historique Git atteste les auteurs des commits, pas à lui seul toutes les tâches faites ensemble ou hors ligne. Cette répartition est à ajuster si elle ne correspond pas au travail réel du binôme.

Un assistant d'IA a aidé à diagnostiquer l'échec de CI du TP6 et à préparer ce brouillon global; les tests, les fichiers du dépôt, les métriques et les résultats d'Actions ont été vérifiés avant rédaction.